Lập trình mạng · Chương 2

Đa luồng: Executor, thread pool và virtual thread

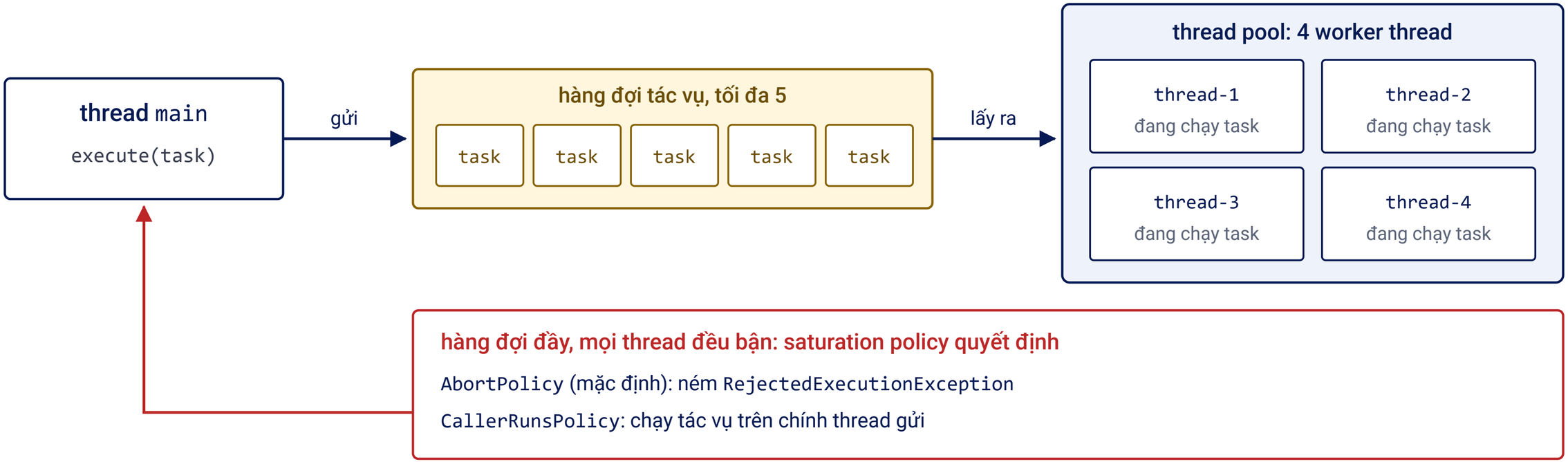
TS. Đặng Ngọc Hùng

Mỗi kết nối một thread mới thì không giới hạn được tài nguyên

1. Mỗi kết nối một thread mới: số thread tăng theo số kết nối, **không có giới hạn**
2. Mỗi **platform thread** chiếm một thread hệ điều hành và một stack riêng: nhiều thread thì cạn bộ nhớ, CPU tốn công chuyển qua lại
3. Cần **tách việc gửi tác vụ khỏi việc chạy tác vụ**, và có quy tắc khi quá tải

> **Câu hỏi của phần:** giới hạn số thread thế nào, làm gì khi quá tải, và phục vụ nhiều kết nối chủ yếu chờ I/O ra sao?

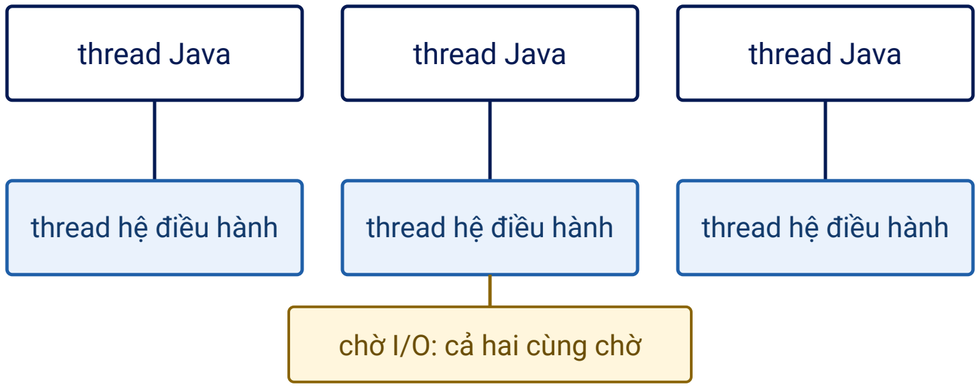
Executor tách gửi tác vụ khỏi chạy tác vụ, thread pool giới hạn số thread



Executor nhận tác vụ, **thread pool** chạy tác vụ bằng một số thread có giới hạn; hàng đợi đầy thì **saturation policy** quyết định.

Virtual thread nhả carrier thread khi chờ I/O

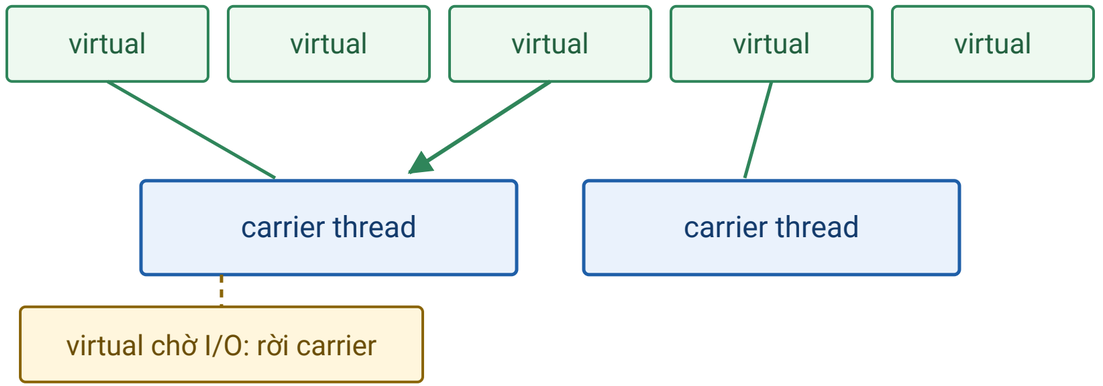
Platform thread



một thread Java giữ một thread hệ điều hành,
kể cả khi chỉ đang chờ

Virtual thread

JVM lập lịch: mỗi lúc một virtual chạy trên một carrier



carrier rảnh chạy virtual thread khác:
hàng nghìn virtual thread chờ I/O trên vài carrier

Virtual thread: JVM lập lịch trên ít **carrier thread**; chờ I/O thì nhả carrier. Hợp với tác vụ **chủ yếu chờ**, không làm phép tính chạy nhanh hơn.

Pool 4 thread, hàng đợi 5 tác vụ: tác vụ thứ 10 chạy trên thread main

Java

```
ThreadPoolExecutor pool = new ThreadPoolExecutor(
        4, 4, 0, TimeUnit.SECONDS,                  // đúng 4 thread
        new ArrayBlockingQueue<>(5),                // hàng đợi tối đa 5 tác vụ
        new ThreadPoolExecutor.CallerRunsPolicy());   // đầy: chạy trên thread gọi execute
for (int i = 1; i <= 10; i++) {
    int task = i;
    pool.execute(() -> work(task));  // gửi tác vụ; pool quyết định thread nào chạy
}
pool.shutdown();  // không nhận tác vụ mới, chạy nốt tác vụ đã nhận
```

Kết quả chạy

```
   6 ms  task 1 runs on pool-1-thread-1
   6 ms  task 2 runs on pool-1-thread-2
   6 ms  task 3 runs on pool-1-thread-3
   6 ms  task 10 runs on main
   6 ms  task 4 runs on pool-1-thread-4
 542 ms  task 6 runs on pool-1-thread-1
 542 ms  task 5 runs on pool-1-thread-4
 542 ms  task 8 runs on pool-1-thread-3
 542 ms  task 7 runs on pool-1-thread-2
1053 ms  task 9 runs on pool-1-thread-1
```

10 000 virtual thread cùng chờ 1 giây, xong trong khoảng 1 giây

Java

```
try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {  // mỗi tác vụ một virtual thread
    for (int i = 0; i < 10_000; i++) {
        executor.submit(() -> {
            carriers.add(carrierOf(Thread.currentThread()));
            Thread.sleep(1_000);  // chờ: virtual thread nhả carrier thread
            return null;
        });
    }
}  // close(): chờ mọi tác vụ xong
```

Kết quả chạy

```
10000 tasks done in 1102 ms
carrier threads used: 8 (CPU cores: 8)
```

Hai cấu hình mặc định của Executors không giới hạn tài nguyên

Java

```
var fixed = (ThreadPoolExecutor) Executors.newFixedThreadPool(4);  // hàng đợi không giới hạn
var cached = (ThreadPoolExecutor) Executors.newCachedThreadPool();  // số thread không giới hạn
Runnable slow = () -> PoolDemo.sleep(1_000);
for (int i = 0; i < 2_000; i++) {
    fixed.execute(slow);
    cached.execute(slow);
}
```

Kết quả chạy

```
fixed:  threads = 4, queued tasks = 1996
cached: threads = 2000, queued tasks = 0
```

- Không pool nào **từ chối** tác vụ: quá tải chỉ lộ ra khi hết bộ nhớ hoặc hết thread
- Dùng `ThreadPoolExecutor`, khai báo rõ **số thread tối đa, dung lượng hàng đợi, saturation policy**

Câu hỏi 1

Thread pool có 4 thread, hàng đợi tối đa 5 tác vụ, saturation policy là CallerRunsPolicy. Thread main gửi liên tiếp 10 tác vụ chạy lâu. Tác vụ thứ 10 chạy trên thread nào?

A. Trên thread đầu tiên của pool xong việc.

B. Trên một thread thứ năm mà pool tạo thêm.

C. Không chạy: bị từ chối với RejectedExecutionException.

D. Trên thread main, thread đã gửi tác vụ.

Câu hỏi 1

Thread pool có 4 thread, hàng đợi tối đa 5 tác vụ, saturation policy là CallerRunsPolicy. Thread main gửi liên tiếp 10 tác vụ chạy lâu. Tác vụ thứ 10 chạy trên thread nào?

A. Trên thread đầu tiên của pool xong việc.

B. Trên một thread thứ năm mà pool tạo thêm.

C. Không chạy: bị từ chối với RejectedExecutionException.

D. Trên thread main, thread đã gửi tác vụ.

Tác vụ 1–4 chiếm 4 thread, 5–9 lấp đầy hàng đợi. Tác vụ 10 gặp hàng đợi đầy và pool đã đủ số thread tối đa, nên saturation policy quyết định: CallerRunsPolicy chạy tác vụ trên thread gửi. Nếu là AbortPolicy (mặc định) thì mới bị từ chối.

Câu hỏi 2

Loại tác vụ nào có lợi nhất khi chuyển từ platform thread sang virtual thread?

A. Tác vụ chủ yếu chờ I/O, như xử lý một kết nối đang chờ client gửi dữ liệu.

B. Tác vụ tính toán nặng, dùng CPU liên tục.

C. Tác vụ cần chạy nhanh hơn trên một lõi CPU.

D. Một tác vụ duy nhất, chạy rất lâu.

Câu hỏi 2

Loại tác vụ nào có lợi nhất khi chuyển từ platform thread sang virtual thread?

A. Tác vụ chủ yếu chờ I/O, như xử lý một kết nối đang chờ client gửi dữ liệu.

B. Tác vụ tính toán nặng, dùng CPU liên tục.

C. Tác vụ cần chạy nhanh hơn trên một lõi CPU.

D. Một tác vụ duy nhất, chạy rất lâu.

Virtual thread nhả carrier thread khi chờ, nên nhiều tác vụ chờ I/O dùng chung vài carrier. Virtual thread không làm phép tính nhanh hơn; tác vụ nặng CPU vẫn bị giới hạn bởi số lõi.

Tổng kết

1. **Executor** tách gửi tác vụ khỏi chạy tác vụ; **thread pool** giới hạn số thread
2. Hàng đợi đầy: **saturation policy** quyết định (từ chối, hoặc chạy trên thread gửi); khai báo rõ bằng `ThreadPoolExecutor`
3. **Virtual thread** chạy trên ít **carrier thread**, nhả carrier khi chờ I/O

Từ khóa cần nhớ Executor · ExecutorService · thread pool · saturation policy · virtual thread · carrier thread

Mở rộng

Chủ đề nâng cao

- Structured concurrency (Java 21, preview)
- Pinning: virtual thread không nhả carrier khi chờ trong khối synchronized

Bài tập mở rộng

Đo thời gian hoàn tất 1 000 tác vụ chờ I/O với pool 10 thread và với virtual thread; giải thích chênh lệch.

Tiếp theo

Bài toán ứng dụng: server tra cứu phục vụ nhiều client